

**МИНОБРНАУКИ РОССИИ**  
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**  
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**  
**Кафедра математического обеспечения и применения ЭВМ**

**ОТЧЕТ**  
**по лабораторной работе №3**  
**по дисциплине «Построение и анализ алгоритмов»**  
**Тема: Расстояние Левенштейна**  
**Вариант 8а**

Студент гр. 4345

Трушкин А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

## **Цель работы**

Изучить и реализовать алгоритм поиска расстояния Левенштейна с учетом дополнительных ограничений на операции вставки и замены для конкретных индексов.

## Постановка задачи

Расстоянием Левенштейна назовём минимальное количество операций вставки одного символа, удаления одного символа и замены одного символа на другой, необходимых для превращения одной строки в другую.

Разработайте программу, осуществляющую поиск расстояния Левенштейна между двумя строками.

Пример:

Для строк `pedestal` и `stien` расстояние Левенштейна равно 7:

- Сначала нужно совершить четыре операции удаления символа:  
`pedestal` -> `stal`.
- Затем необходимо заменить два последних символа: `stal` -> `stie`.
- Потом нужно добавить символ в конец строки: `stie` -> `stien`.

Параметры входных данных:

Первая строка входных данных содержит строку из строчных латинских букв. ( $S, 1 \leq |S| \leq 2550$ ).

Вторая строка входных данных содержит строку из строчных латинских букв. ( $T, 1 \leq |T| \leq 2550$ ).

Параметры выходных данных:

Одно число  $L$ , равное расстоянию Левенштейна между строками  $S$  и  $T$ .

Sample Input:

`pedestal`

`stien`

Sample Output:

7

**Вариант 8а** - "Неновые элементы второй строки": на вход дополнительно подаётся список индексов 2-ой строки, элементы по которым не должны быть получены вставкой (т.е. могут быть получены только сохранением или заменой), но если "неновым" оказался символ "К", то заменой его получать тоже нельзя, можно только сохранением.

## Выполнение работы

Для реализации алгоритма вычисления расстояния Левенштейна используется матрица размера  $n \times m$ .

В общем случае на первом шаге первый столбец заполняется ценой операции удаления каждого символа (0, 1, 2, ...,  $n$ ), а первая строка — ценой операции вставки (0, 1, 2, ...,  $m$ ). Каждая ячейка  $dp[i][j]$  матрицы соответствует минимальной стоимости превращения префикса строки  $A$  длиной  $i$  в префикс строки  $B$  длиной  $j$ .

Далее, при заполнении внутренних ячеек, программа анализирует три возможных сценария перехода из предыдущих состояний:

- При совпадении символов: значение ячейки  $dp[i][j]$  принимается равным значению по диагонали ( $dp[i-1][j-1]$ ), так как в данном случае операция не требуется и стоимость не увеличивается.
- При несовпадении символов: значение вычисляется следующим образом:  $\min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1$ . Это означает выбор наилучшего пути из трех возможных операций: удаления, вставки или замены.

В соответствии с Вариантом 8а, классический алгоритм был модифицирован для учета «неновых» элементов второй строки ( $B$ ). В стандартную логику добавляются следующие правила:

- Запрет вставки: Если индекс символа в строке  $B$  входит в список «запрещенных», операция `insert` для этого столбца становится невозможной, ей присваивается значение бесконечности  $\infty$ .
- Особое правило для символа "К": Если символ на запрещенном индексе является буквой «К», его запрещено получать не только вставкой, но и операцией замены (`replace`). Таким образом, такой символ может быть получен только путем сохранения (`match`), если он уже присутствовал в исходной строке  $A$  на соответствующей позиции.

Для выполнения требования по отслеживанию хода алгоритма, программа осуществляет построчный вывод матрицы на каждом этапе вычислений. Это позволяет наглядно увидеть, как ограничения варианта 8а

создают «непроходимые» участки в таблице и как алгоритм находит альтернативные пути с минимальной стоимостью.

Сложность алгоритма по времени  $O(N*M)$ , где  $N$  и  $M$  – длины строк.

Сложность алгоритма по памяти  $O(N*M)$ , обусловлена необходимостью хранения и вывода полной матрицы для демонстрации промежуточных результатов.

Исходный код программы см. в приложении А.

Результаты тестирования см. в Таблице 1.

Таблица 1 – Результаты тестирования

| № | Входные данные        | Выходные данные | Комментарий                                                 |
|---|-----------------------|-----------------|-------------------------------------------------------------|
| 1 | abc<br>aKc<br>1       | Невозможно      | Букву 'K' на запрещенном месте нельзя вставить или заменить |
| 2 | aKc<br>aKc<br>1       | 0               | 'K' совпадает, операция match разрешена                     |
| 3 | map<br>mad<br>2       | 1               | Замена на 'd' разрешена, так как это не 'K'                 |
| 4 | axK<br>aK<br>1        | 1               | Обход запрета через удаление 'x' для совпадения 'K'         |
| 5 | (пусто)<br>apple<br>0 | Невозможно      | Запрет вставки в самом начале разрывает путь                |

## **Выводы**

В ходе работы был реализован модифицированный алгоритм поиска редакционного расстояния. Внедрение ограничений Варианта 8а позволило продемонстрировать гибкость динамического программирования: использование бесконечности эффективно исключает недопустимые пути трансформации строки. Пошаговый вывод матрицы подтвердил корректность работы логики запретов при обработке специфических символов и индексов.

## ПРИЛОЖЕНИЕ А

### ИСХОДНЫЙ КОД ПРОГРАММЫ

#### Файл main.py

```
def print_dp(dp, s1, s2):
    header = "          " + " ".join(f"{char:>3}" for char in s2)
    print(header)
    for i in range(len(dp)):
        char_a = s1[i-1] if i > 0 else "#"
        row = []
        for j in range(len(dp[0])):
            val = dp[i][j]
            display = " ∞ " if val >= 9999 else f"{val:3}"
            row.append(display)
        print(f"{char_a} | " + " ".join(row))
    print()

def levenstein_8a(str_1, str_2, forbidden):
    n, m = len(str_1), len(str_2)
    INF = 9999
    dp = [[INF] * (m + 1) for _ in range(n + 1)]
    dp[0][0] = 0

    for i in range(1, n + 1):
        dp[i][0] = i
    for j in range(1, m + 1):
        if (j - 1) in forbidden:
            print(f"(!) Вставка на индексе {j-1} запрещена, прерываем
первую строку.")
            break
        dp[0][j] = j

    print("\nМатрица после инициализации границ:")
    print_dp(dp, str_1, str_2)

    print("Построчное заполнение:")
    for i in range(1, n + 1):
        for j in range(1, m + 1):
```

```

        delete = dp[i-1][j] + 1

        if (j - 1) in forbidden:
            insert = INF
        else:
            insert = dp[i][j-1] + 1

        if str_1[i-1] == str_2[j-1]:
            replace = dp[i-1][j-1]
        else:
            if (j - 1) in forbidden and (str_2[j-1] == 'K' or
str_2[j-1] == 'k'):
                replace = INF
            else:
                replace = dp[i-1][j-1] + 1

        dp[i][j] = min(delete, insert, replace)

    print(f"Шаг {i} (символ A: '{str_1[i-1]}'):")
    print_dp(dp[:i+1], str_1[:i], str_2)

return dp[n][m]

string_a = input("Строка A: ").strip()
string_b = input("Строка B: ").strip()
forbidden_indices = list(map(int, input("Индексы запрета: ").split()))

result = levenstein_8a(string_a, string_b, forbidden_indices)

if result >= 9999:
    print("Результат: Невозможно")
else:
    print(f"Итоговое расстояние: {result}")

```